

Chapter 5: Computer Arithmetic

Prepared By Er Abhishek Singh
Computer Engineer
Department Of IT
Government of Nepal

Computer Arithmetic

- Arithmetic Operations (Basic Operations):
 - Addition
 - Subtraction
 - Multiplication
 - Division
- With CPU:
 - **ALU** *does all the above operations* and other logical and shift operations
 - CU *generates necessary signals* and hence take cares of ALU operations
 - Registers *hold the data temporarily* to assist the operations especially during execution of the operations (i.e. instructions)
- Each operation can be explained via algorithms
- Best (*and Simple to learn*) algorithms are picked and explained here!

Arithmetic Logic Unit (ALU)

The ALU is a digital circuit within the CPU that performs all arithmetic and logical operations. It takes input from registers, processes the operation, and sends the result back to a destination register or memory.

- Performs operations like addition, subtraction, AND, OR, NOT, etc.
- Controlled by the control unit based on the instruction type.

Functions and Operations of ALU

1. Arithmetic Operations

ALU performs arithmetic operations to carry out basic mathematical calculations on binary numbers.

- Operations include addition, subtraction, increment, and decrement.
- Used during instruction execution for tasks like ADD, SUB, INC, and DEC.

2. Logical Operations

These operations manipulate data at the bit level using logic gates.

- Includes bitwise operations like AND, OR, XOR, and NOT.
- Essential for masking, setting, clearing, or toggling specific bits.

Arithmetic Circuit

The basic arithmetic micro operations (addition, subtraction, increment and

Decrement) can be performed in one composite arithmetic circuit

Select			Input	Output	Micro operation
S_1	S_0	C_{in}	Y	$D = A + Y + C_{in}$	
0	0	0	B	$D = A + B$	Add
0	0	1	B	$D = A + B + 1$	Add with Carry
0	1	0	$\bar{B}$	$D = A + \bar{B}$	Subtract with Borrow
0	1	1	$\bar{B}$	$D = A + \bar{B} + 1$	Subtract
1	0	0	0	$D = A$	Transfer A
1	0	1	0	$D = A + 1$	Increment A
1	1	0	1	$D = A - 1$	Decrement A
1	1	1	1	$D = A$	Transfer A

Working this arithmetic circuit can perform 8 operations among them some are: -
Addition:-

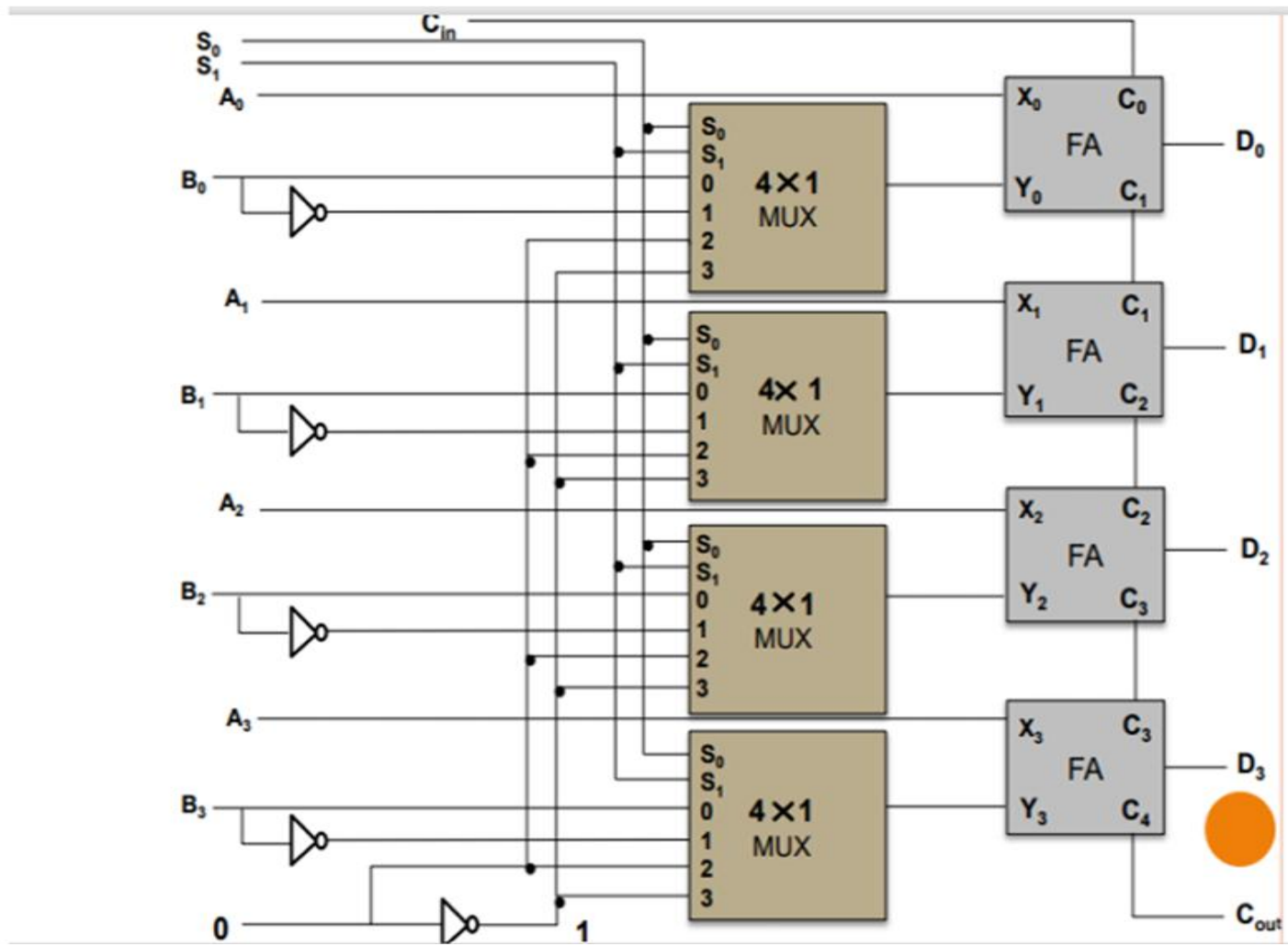
- When $S_1 S_0 = 0 0$, the value of B is applied to the Y inputs of the adder.
- If $C_{in} = 0$, the output $D = A + B$.
- If $C_{in} = 1$, output $D = A + B + 1$.
- Both cases perform the add micro operation with or without adding the input carry.

Subtraction

- When $S_1 S_0 = 0 1$, the value of B is applied to the Y inputs of the adder.
- If $C_{in} = 1$, then $D = A + B + 1$. this produces A plus the 2's complement of B, which is equivalent to a subtraction of $A - B$.
- When $C_{in} = 0$, then $D = A + B$. this is equivalent to a subtract with borrow, that is, $A - B - 1$.

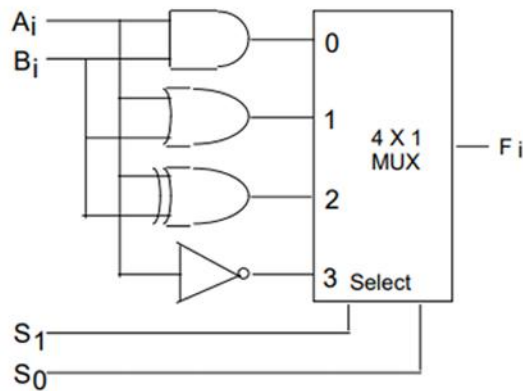
Increment

- When $S_1 S_0 = 1 0$, the inputs from B are neglected, and instead, all 0's are inserted into the y inputs. The output becomes $D = A + 0 + C_{in}$.
- This gives $D = A$ when $C_{in} = 0$ and $D = A + 1$ when $C_{in} = 1$.
- In the first case we have a direct transfer from the input A to output D and in the second case, the value of A is incremented by 1.



S_1	S_0	Output	Operation
0	0	$E = A \wedge B$	AND
0	1	$E = A \vee B$	OR
1	0	$E = A \oplus B$	XOR
1	1	$E = \bar{A}$	Complement

(b) Function table



Logic Micro operations

- Logic micro operations are bit-wise operations, i.e., they work on the individual bits of data.
- It is useful for bit manipulations on binary data.
- Useful for making logical decisions based on the bit value. There are, in principle, 16 different logic functions that can be defined over two binary input variables.
- However, most systems only implement four of these – AND , OR , XOR and Complement/NOT.

Hardware implementation

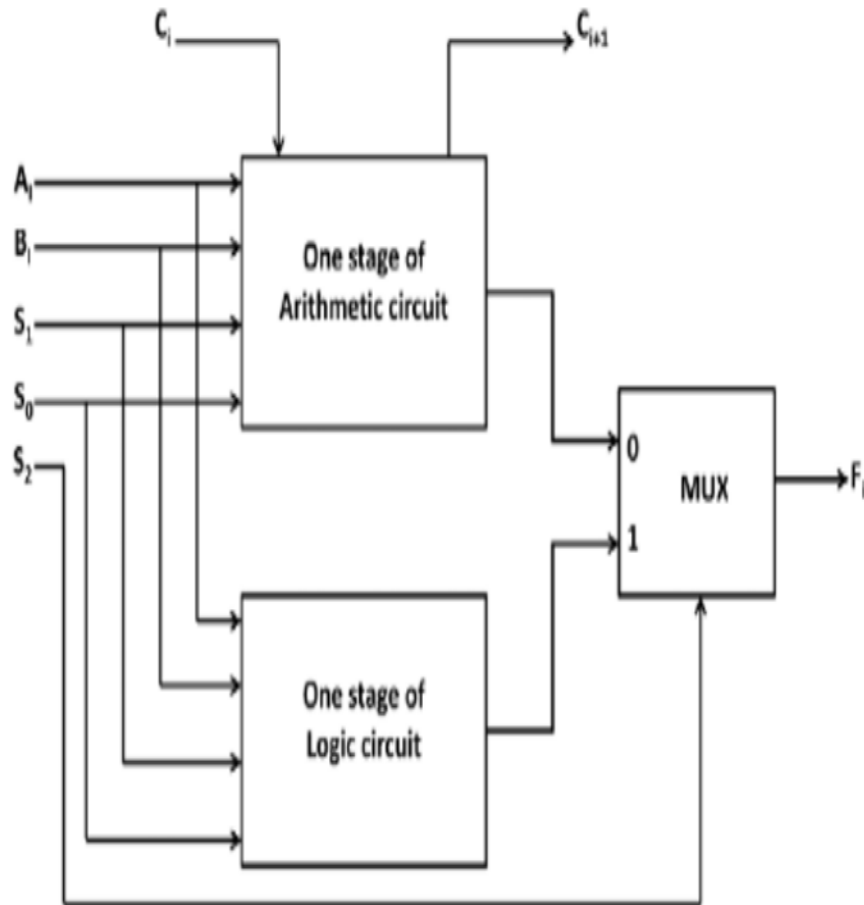
- Hardware implementation of logic micro operations requires that logic gates be inserted be each bit or pair of bits in the registers to perform the required logic operation.

ALU DESIGN

The ALU can be constructed by linking the Arithmetic Circuit and the Logic Circuit

Discussed in the previous two sections.

- A third select line S3 is used to select either arithmetic or a logic operation at the Output of the ALU.
- Although the two circuits can be combined in this manner, this may not be the best way to design an ALU.
- The complete block diagram of the one bit ALU



One stage means this circuit processes **one bit** of the binary inputs A_i and B_i and produces **one bit** of the output F_i .

The stage has two main parts:

One stage of Arithmetic circuit

One stage of Logic circuit

Arithmetic circuit part performs bitwise arithmetic operations (like add, subtract, increment, decrement) based on control signals S_1, S_0 and possibly C_m (carry-in from the previous stage).

Logic circuit part performs bitwise logical operations (like AND, OR, XOR, NOT) based on S_1 and S_0 .

Multiplexer (MUX) is used to select whether the output comes from the arithmetic unit or the logic unit.

The selection is controlled by S_2 .

When $S_2=0$ the output from the arithmetic circuit is chosen.

When $S_2=1$ the output from the logic circuit is chosen.

Control signals:

S_1 choose the operation within each unit.

S_2 chooses between **arithmetic mode** ($S_2=0$) and **logic mode** ($S_2=1$).

Carry handling:

In arithmetic mode, a **carry-out** is generated and passed to the next stage.

In logic mode, carry is usually ignored.

This single-bit stage is replicated for each bit position to form a full multi-bit ALU (e.g., 4-bit, 8-bit, 16-bit).

Overall function:

$S_2 = 0 \rightarrow$ Arithmetic operations (with carry chain between stages)

$S_2 = 1 \rightarrow$ Logic operations (bitwise, no carry propagation)

Selection				Output	Function
S_2	S_1	S_0	C_{in}		
0	0	0	0	$F = A$	Transfer A
0	0	0	1	$F = A + 1$	Increment A
0	0	1	0	$F = A + B$	Addition
0	0	1	1	$F = A + B + 1$	Addition with Carry
0	1	0	0	$F = A - B - 1$	Subtraction with Borrow
0	1	0	1	$F = A - B$	Subtraction
0	1	1	0	$F = A - 1$	Decrement A
0	1	1	1	$F = A$	Transfer A
1	0	0	X	$F = A + B$	OR
1	0	1	X	$F = A \oplus B$	XOR
1	1	0	X	$F = AB$	AND
1	1	1	X	$F = A'$	NOT

The table defines a multi-function unit controlled by 4 selection bits: S_2, S_1, S_0 , where C_m is likely a carry-in or mode control bit.

When $S_2=0$, the unit performs arithmetic operations — addition, subtraction, increment, decrement, and transfers — and C_m affects the result in these modes.

When $S_2=1$, the unit performs logical operations — AND, OR, XOR, NOT — and C_m is ignored (marked as X don't care).

Transfer A occurs in two cases: (0,0,0,0) and (0,1,1,1) showing that the same output can be selected by different control codes.

Increment A is performed with control (0,0,0,1), which adds 1 to input A.

Addition of A and B is selected with (0,0,1,0) adding $C_m=1$ gives $A+B+1$ (addition with carry).

Subtraction operations include:

$A-B-1$ when (0,1,0,0) (subtract with borrow)

$A-B$ when (0,1,0,1) (direct subtraction)

Decrement A is achieved with (0,1,1,0), giving $A-1$

Logical functions when $S_2=1$

$S_1S_0=00$ $F=A+B$ (logical OR)

$S_1S_0=01$ $F=A \oplus B$ (XOR)

$S_1S_0=10$ $F=A \cdot B$ (AND)

$S_1S_0=11$ $F=A'$ (NOT)

Signed Number Representation

128	64	32	16	8	4	2	1
-----	----	----	----	---	---	---	---

- Sign Magnitude Form: +41

0	1	0	1	0	0	1
---	---	---	---	---	---	---

 [Using MSB as sign bit] - 41

1	1	0	1	0	0	1
---	---	---	---	---	---	---
- 1's Complement Form: +41

0	1	0	1	0	0	1
---	---	---	---	---	---	---

 [flipping the all bits] -41

1	0	1	0	1	1	0
---	---	---	---	---	---	---
- 2's Complement Form: +41

0	1	0	1	0	0	1
---	---	---	---	---	---	---

 [1's Complement +1] -41

1	0	1	0	1	1	1
---	---	---	---	---	---	---

Addition and Subtraction

- TWO numbers (A and B)
- Several Cases (Plus N Minus)
 - 4 Operations for Addition
 - 4 Operations for Subtraction

Needs Plus Sign in Between

A at first and Minus Sign in Between

B at first and Minus Sign in Between

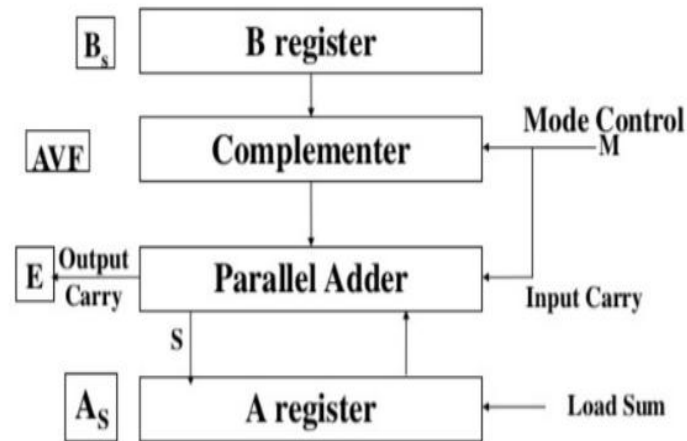
No Sign with Zero

Addition: $A + B$; A: Augend; B: Addend
Subtraction: $A - B$; A: Minuend; B: Subtrahend

Operation	Add Magnitude	Subtract Magnitude		
		When $A > B$	When $A < B$	When $A = B$
$(+A) + (+B)$	$+(A + B)$			
$(+A) + (-B)$		$+(A - B)$	$-(B - A)$	$+(A - B)$
$(-A) + (+B)$		$-(A - B)$	$+(B - A)$	$+(A - B)$
$(-A) + (-B)$	$-(A + B)$			
$(+A) - (+B)$		$+(A - B)$	$-(B - A)$	$+(A - B)$
$(+A) - (-B)$	$+(A + B)$			
$(-A) - (+B)$	$-(A + B)$			
$(-A) - (-B)$		$-(A - B)$	$+(B - A)$	$+(A - B)$

Addition and Subtraction

Hardware Implementation



Hardware for signed-magnitude addition and subtraction

Let A and B be two registers that hold the magnitudes of the numbers
A_s and B_s be two flip-flops, which hold the corresponding signs
Parallel adder is needed to perform the microOperation A + B
The Complementer for generating the 2's Complement while performing subtraction operation

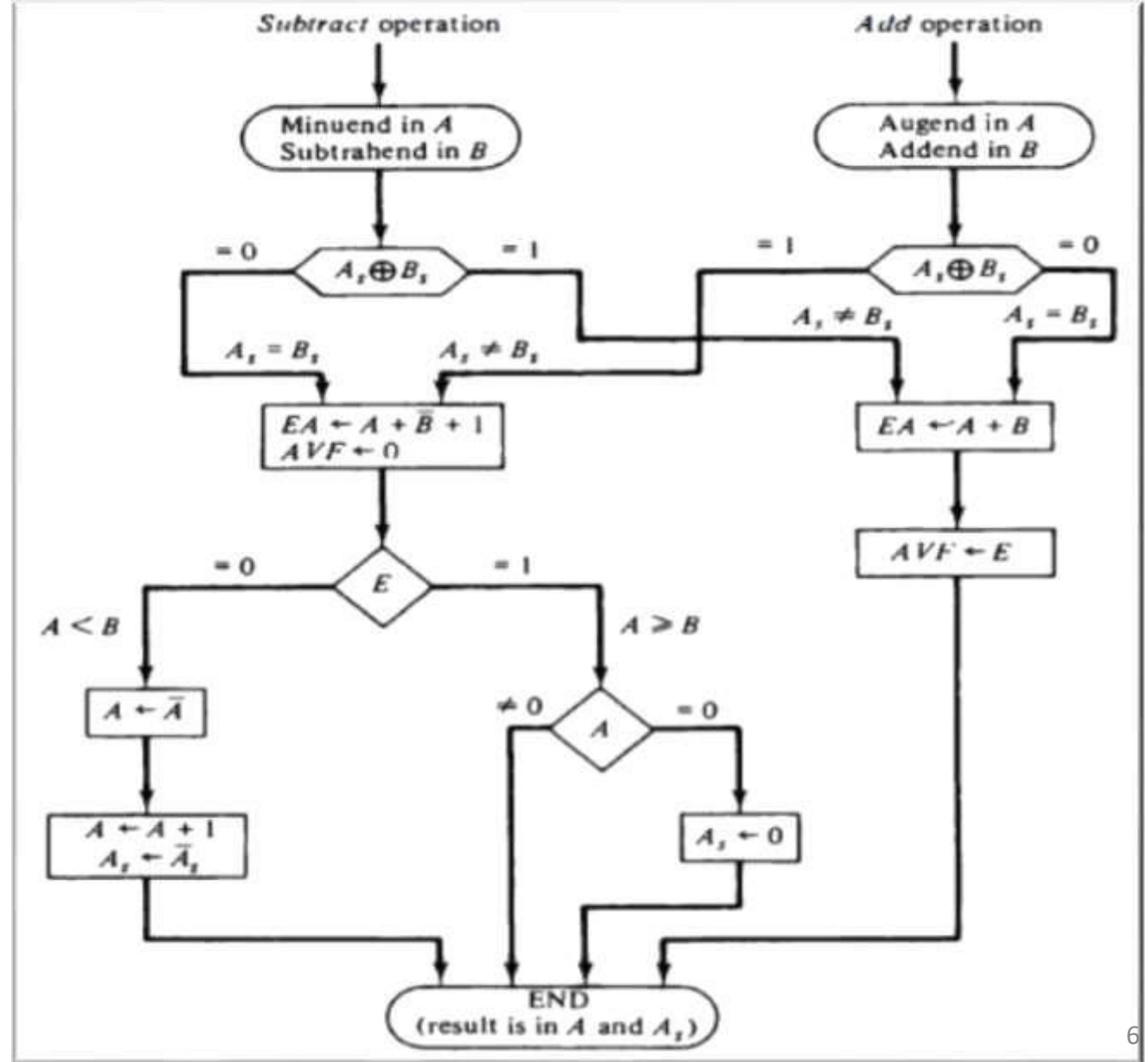
M is Mode of operation:

When M = 0, the output of B is transferred to the adder, the input carry is 0, and the adder is equal to the sum A + B

When M = 1, the 1's complement of B is applied to the adder, the input carry is 1, and output is = A + B + 1 (this is equal to A plus the 2's complement of B, which is equivalent to the A - B)

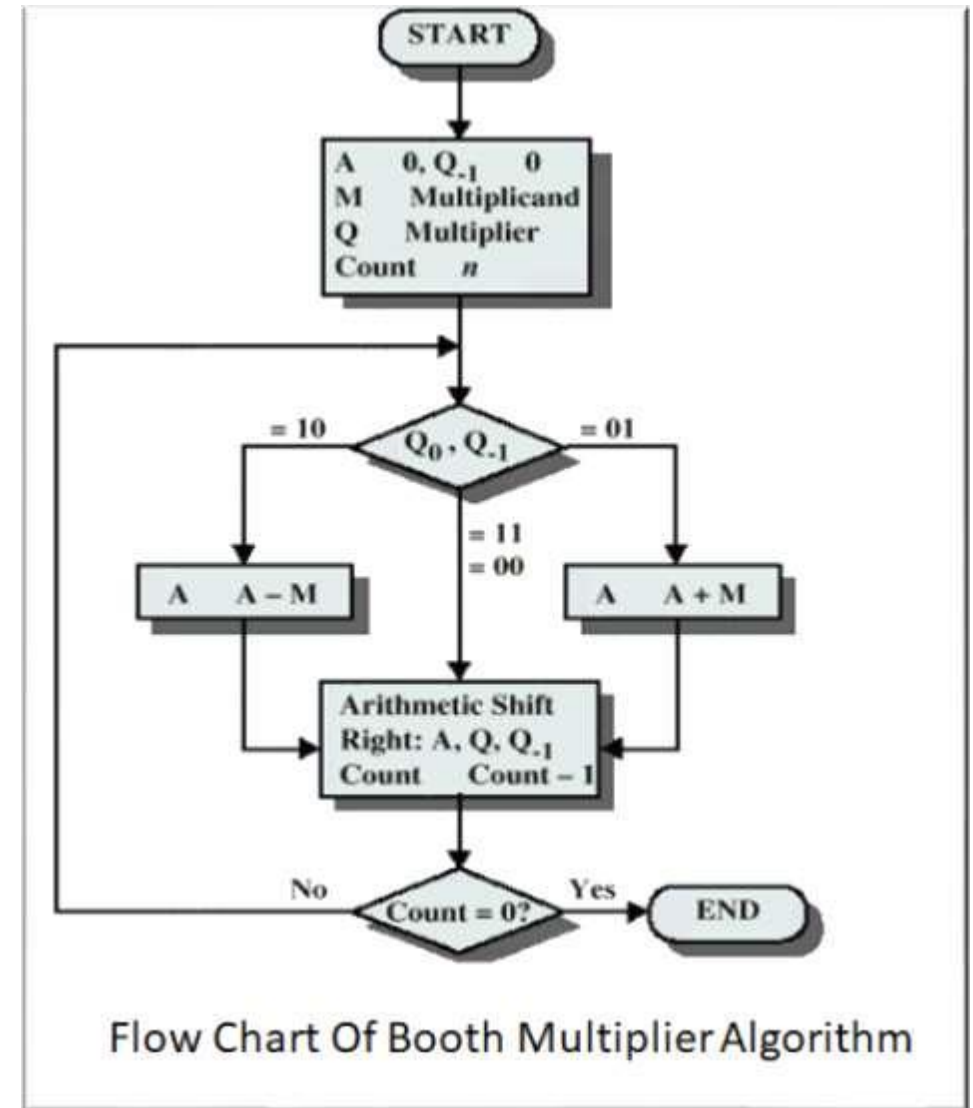
Addition and Subtraction [Flow Chart]

- For Subtraction:
 - A is Minuend
 - B is Subtrahend
- For Addition:
 - A is Augend
 - B is Addend
- First comparing the Sign Bit
 - $A_s = B_s$ means same sign
 - $(A_s \text{ X-OR } B_s = 0)$
- EA: Extended Accumulator
- $AVF \leftarrow E$
 - (overflow bit is stored in addition)
- $AVF \leftarrow 0$
 - (there is no overflow in subtraction)
- There are two options in subtraction
 - $A \geq B$ and $A < B$



Signed Number Multiplication [Booth's Algorithm]

- Multiplicand (M) [$M_3M_2M_1M_0$] = $(-7)_{10}$
 - Multiplier (Q) [$Q_3Q_2Q_1Q_0$] = $(+3)_{10}$
 - Q_{-1} : an extra (single) bit of Multiplier
 - Count (n) = Number of Bits (4 here)
 - Accumulator (A): Initialized as 0000
 - Result= AQ
- { M= -7 (i.e. 2's Compliment of $(0111)_2$) => $1000+1 = (1001)_2$
If we are talking about $-M$; $-M = (0111)_2$ [$= (7)_{10}$] }

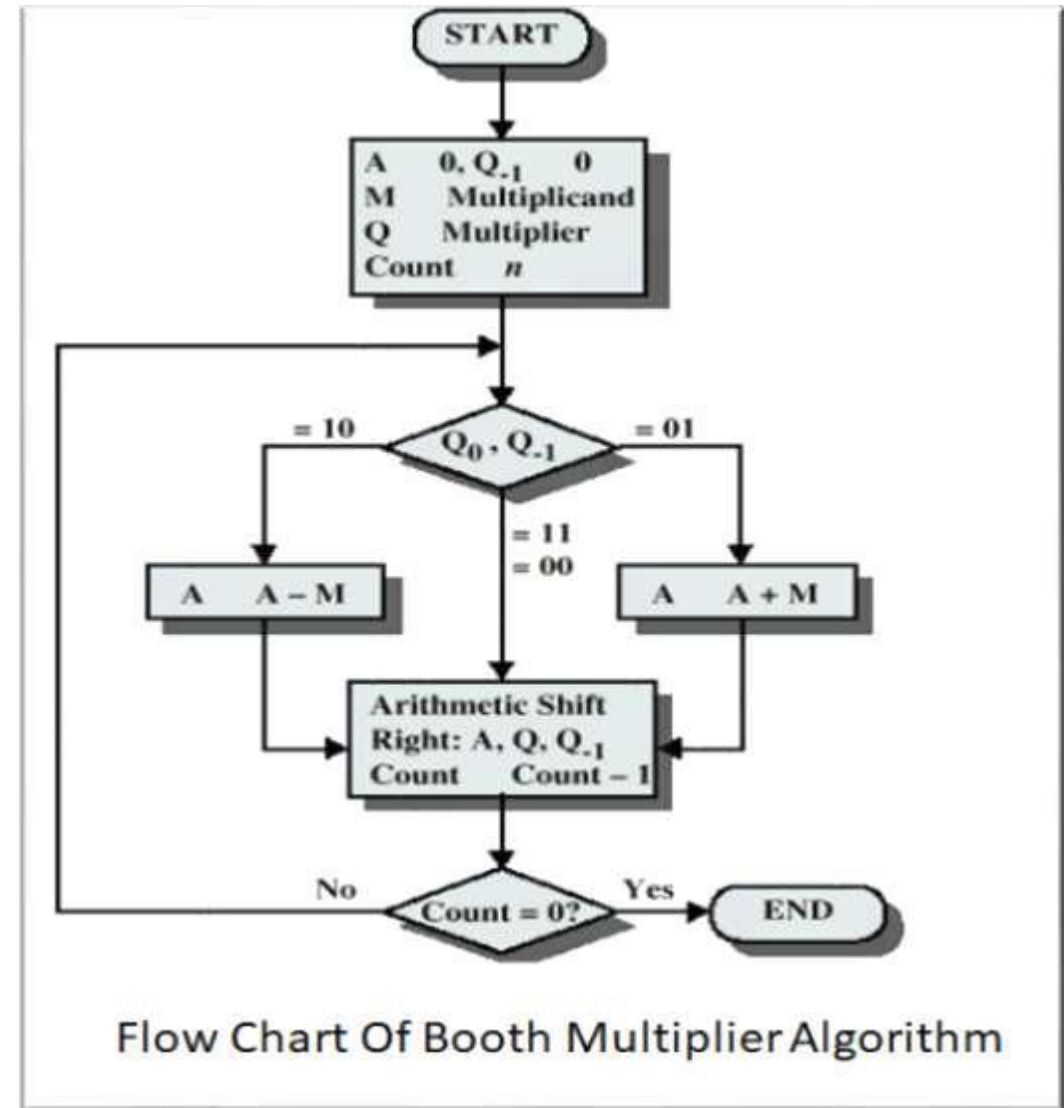


Signed Number Multiplication [Booth's Algorithm]

- Tracing Table:

n	A	Q	Q_{-1}	Action
4	0000	0011	0	Initialization
	0111	0011	0	$A = A - M$
3	0011	1001	1	ASR AQQ ₋₁
2	0001	1100	1	ASR AQQ ₋₁ Lost
	1010	1100	1	$A = A + M$
1	1101	0110	0	ASR AQQ ₋₁
0	1110	1011	0	ASR AQQ ₋₁

Now Result is in AQ = 11101011
 MSB= 1 means negative number
 Needs 2's compliment calculation
 $00010100 + 1 = 00010101 (= -21)$



Division (Restoring & Non-Restoring)

Base 10 (Decimal Format)

14 Quotient

Divisor 12) 169 Dividend
-12

49 Partial Remainder
-48

1 Remainder

Base 2 (Binary Format)

12 = 1100 || 169 = 10101001

00001110
1100) 10101001
-1100

010010
- 1100

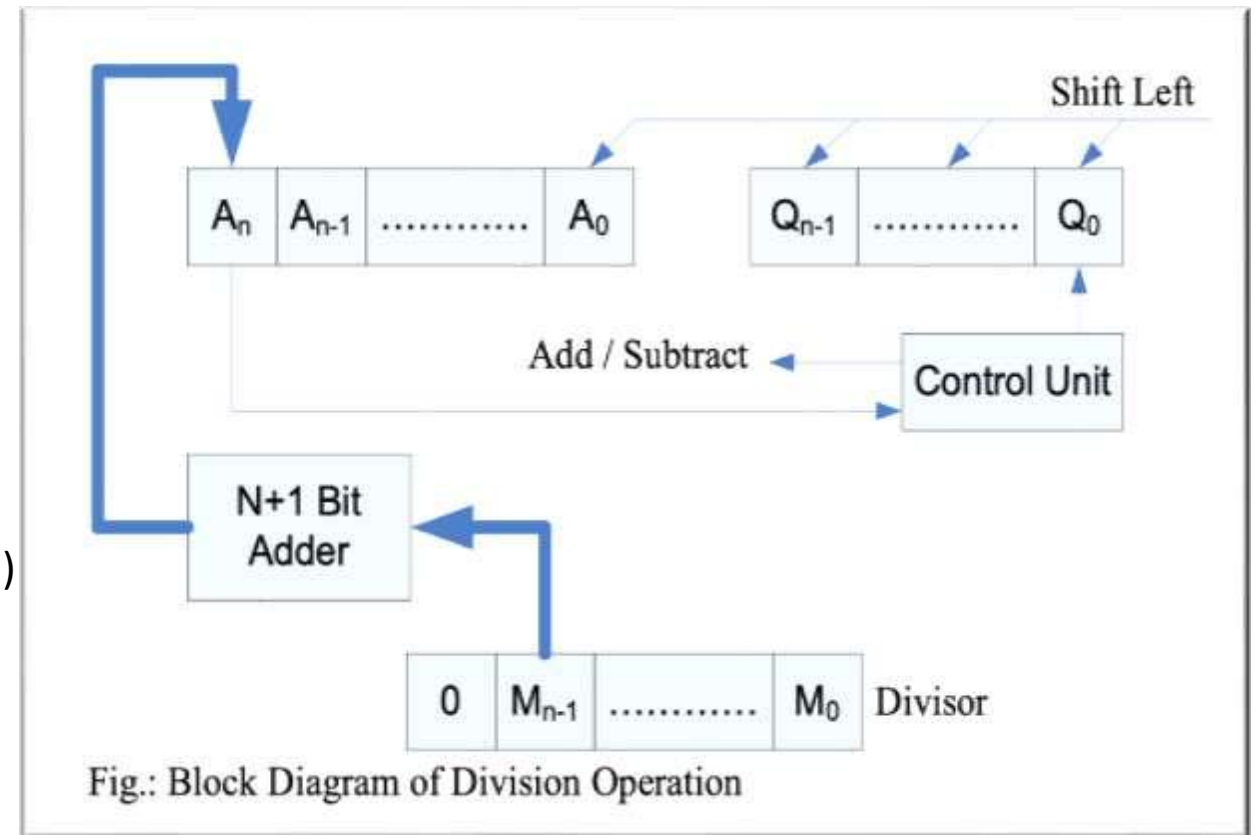
001100
- 1100

00001
- 0000

0001

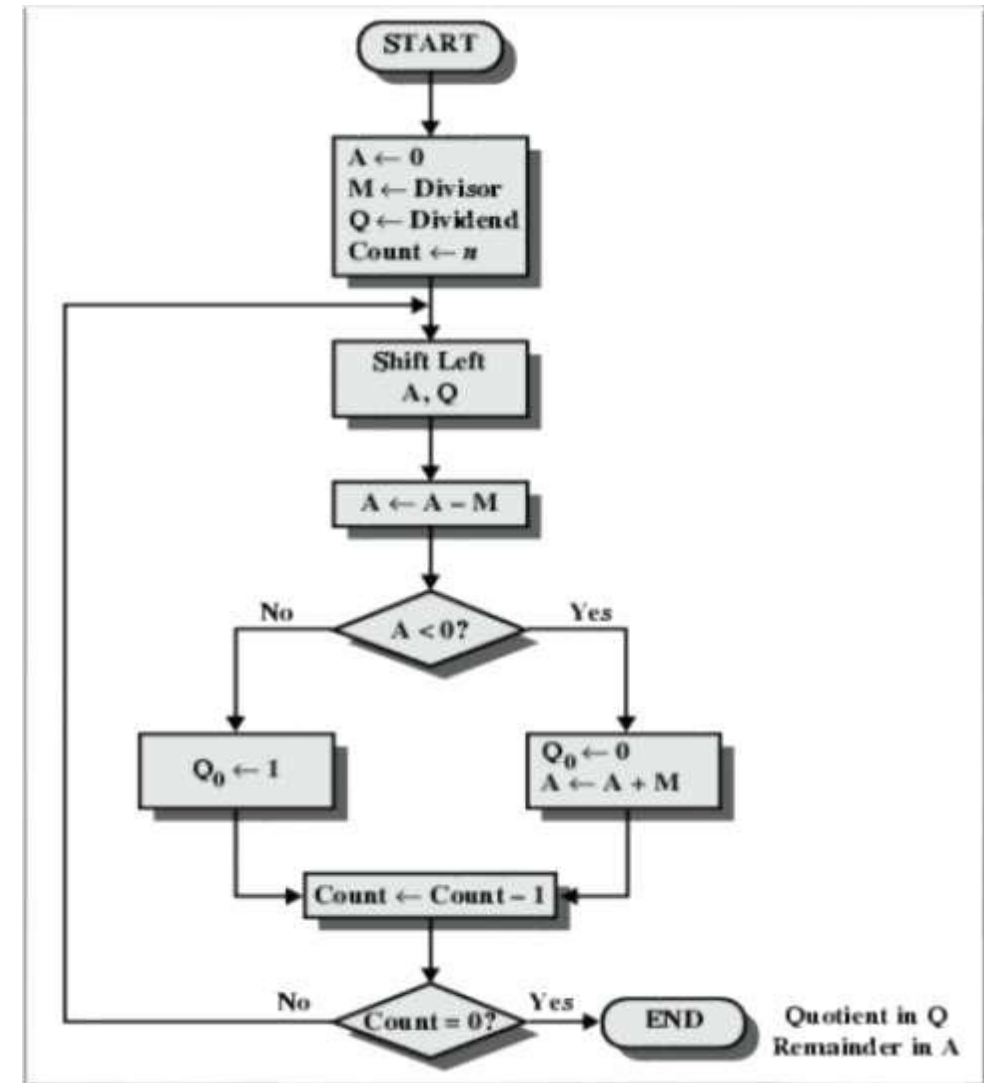
Division (Restoring & Non-Restoring)

- THREE Registers are in used:
 - A- Accumulator (initialized as Zero)
 - Q- Dividend
 - M- Divisor
- A and M are of (n+1) sized
 - So, using (n+1) adder here
- CU generates the control logic (add, subtract, shift)
 - Addition means addition straight forward
 - Subtraction is addition with 2's complement (of M)
 - Left shift used here
- Q_0 value is determined as per A_n value



Division (Restoring)

- Step 1:
 - Initialize A, Q, & M registers to zero
 - dividend and divisor as it is
 - counter to n (number of bits of dividend)
- Step 2:
 - Shift A, Q left one binary position
- Step 3:
 - Subtract M from A placing answer back in A
 - If sign of A is 1, set Q_0 to 0
 - Add M to A and restore to A
 - If sign of A is 0, set Q_0 to 1
- Step 4:
 - Decrease counter
 - if counter > 0, repeat process from step 2
 - else STOP the process
- The final Answer:
 - **Remainder will be in A**
 - **Quotient will be in Q**



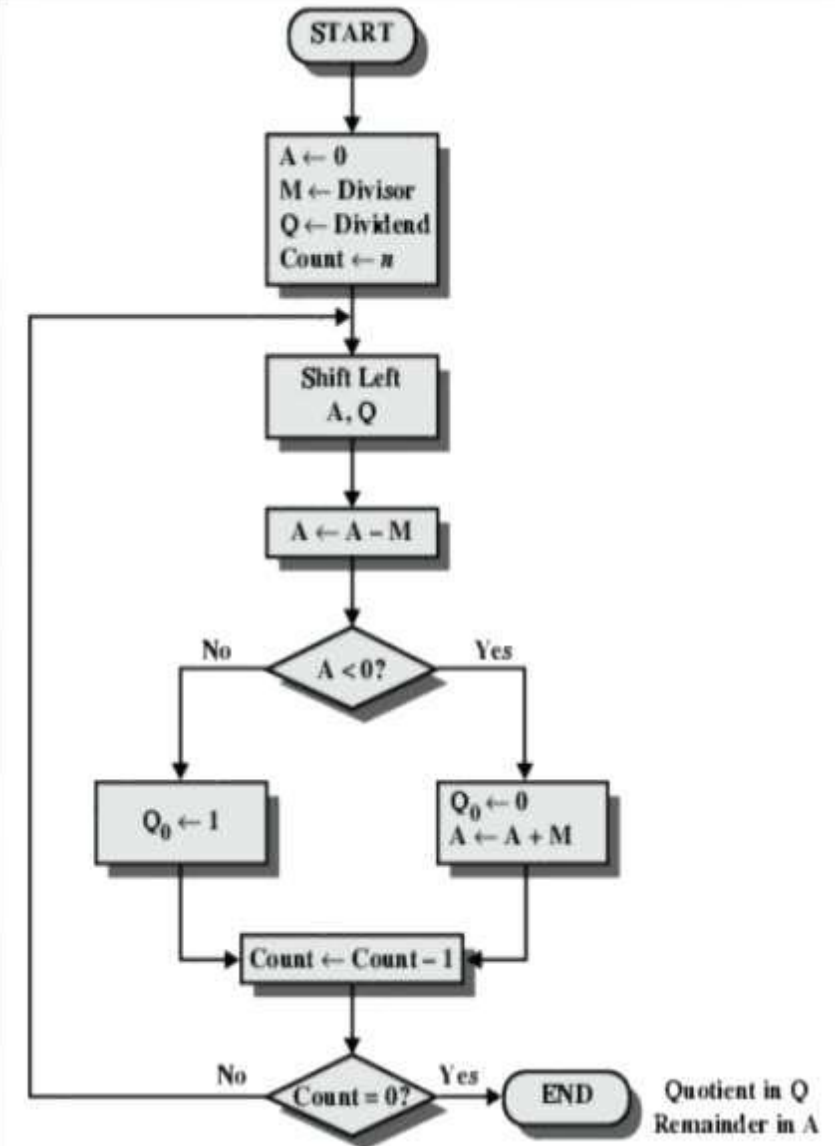
Division (Restoring)

Example: Divide 15 (1111) by 4 (0100)

A	Q	M	$\overline{M}+1$	Count	Remarks
00000	1111	00100	11100	4	Initialization
00001 11101 00001	111□ 111□ 1110	- - -	- - -	- - 3	Shift Left A, Q Sub ($A \leftarrow A - M$) $Q_0 \leftarrow 0$, Add ($A \leftarrow A + M$)
00011 11111 00011	110□ 110□ 1100	- - -	- - -	- - 2	Shift Left A, Q Sub ($A \leftarrow A - M$) $Q_0 \leftarrow 0$, Add ($A \leftarrow A + M$)
00111 00011 00011	100□ 100□ 1001	- - -	- - -	- - 1	Shift Left A, Q Sub ($A \leftarrow A - M$) Set $Q_0 \leftarrow 1$
00111 00011 00011	001□ 001□ 0011	- - -	- - -	- - 0	Shift Left A, Q Sub ($A \leftarrow A - M$) Set $Q_0 \leftarrow 1$

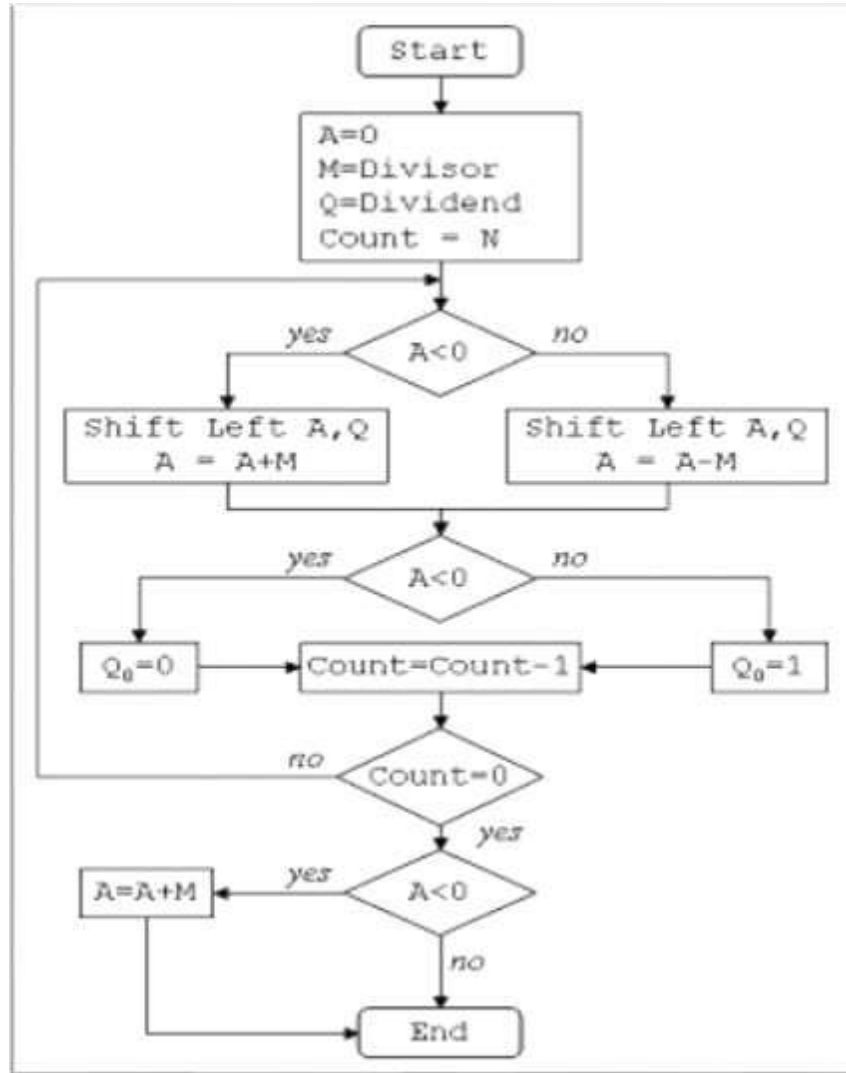
Quotient in Q = 0011 = 3

Remainder in A = 00011 = 3



Division (Non-Restoring)

- Step 1:
 - Initialize $A = 0$
 - Q (dividend) as given
 - M (divisor) as given
- Step 2:
 - Check sign bit of A
 - If $A < 0$ (i.e. Negative Value)
 - Shift A, Q left one binary position
 - $A = A + M$
 - If $A \geq 0$ (i.e. Positive Value)
 - Shift A, Q left one binary position
 - $A = A - M$
- Step 3:
 - If sign of A is 0
 - set Q_0 to 1 else set Q_0 to 0
- Step 4:
 - Decrease counter
 - If counter > 0
 - repeat from step 2
 - else go to step 5
- Step 5:
 - If $A \geq 0$ (i.e. positive)
 - **content of A is remainder**
 - else **Add M to A to get the remainder**
 - **The quotient will be in Q .**



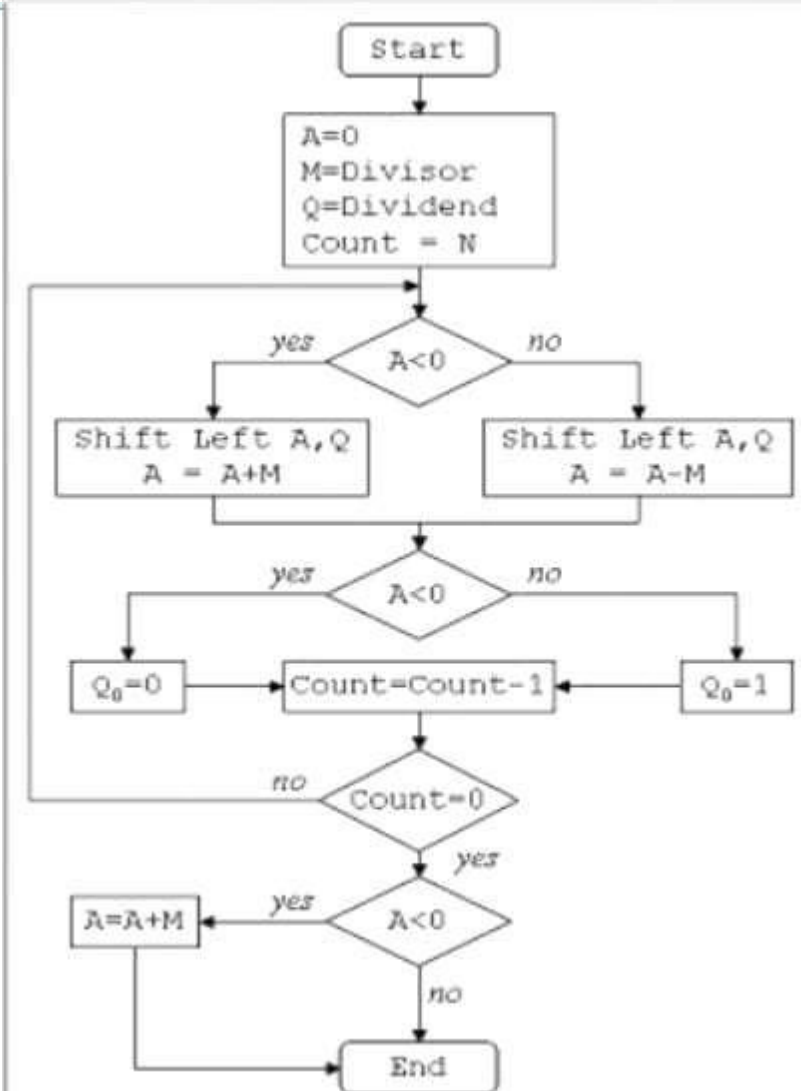
Division (Non-Restoring)

Example: Divide 1110 (14) by 0011 (3) using non-restoring division.

A	Q	M	$\overline{M}+1$	Count	Remarks
00000	1110	00011	11101	4	Initialization
00001 11110 11110	110□ 110□ 1100	- - -	- - -	- - 3	Shift Left A, Q Sub ($A \leftarrow A - M$) Set Q_0 to 0
11101 00000 00000	100□ 100□ 1001	- - -	- - -	- - 2	Shift Left A, Q Add ($A \leftarrow A + M$) Set Q_0 to 1
00001 11110 11110	001□ 001□ 0010	- - -	- - -	- - 1	Shift Left A, Q Sub ($A \leftarrow A - M$) Set Q_0 to 0
11100 11111 11111	010□ 010□ 0100	- - -	- - -	- - 0	Shift Left A, Q Add ($A \leftarrow A + M$) Set Q_0 to 0
00010	0100	-	-	-	Add ($A \leftarrow A + M$)

Quotient in Q = 0011 = 3

Remainder in A = 00010 = 2



with Floating-Point Numbers ...

- The floating point representation of the number has two parts. The first part represents a signed fixed point numbers called mantissa or significand. The second part designates the position of the decimal (or binary) point and is called exponent.
- For: + 6132.789

Fraction	Exponent
+0.6132789	+04

(This representation is equivalent to the scientific notation $+0.6132789 \times 10^{+4}$)
- The floating point is always interpreted to represent a number in the following form $\pm M \times R^{\pm E}$.
- Only the mantissa M and the exponent E are physically represented in the register (including their sign).
- The radix R and the radix point position of the mantissa are always assumed.
- A floating point binary no is represented in similar manner except that it uses base 2 for the exponent.
- For: 0.1001110×2^{100}

Fraction	Exponent
01001110	000100

Issues with Floating-Point Numbers Arithmetic

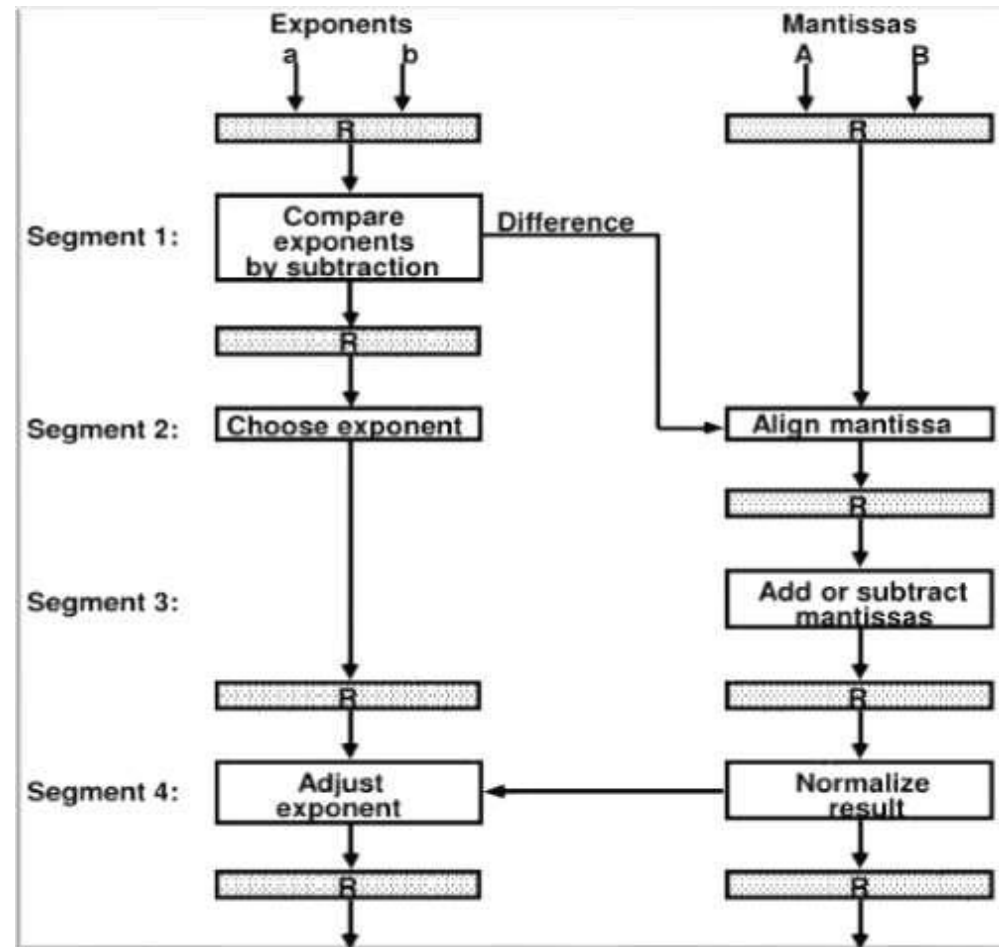
- Exponent Overflow:
 - A positive exponent exceeds the maximum possible exponent value.
- Exponent Underflow:
 - A negative exponent which is less than the minimum possible value.
- Significand Overflow:
 - The addition of two significands of the same sign may carry in a carry out of the most significant bit.
- Significand underflow:
 - In the process of aligning significands, digits may flow off the right end of the significand.

Floating Point Number Addition/Subtraction

- Floating-point adder/subtractor
- [1] Compare the exponents
- [2] Align the mantissa
- [3] Add/sub the mantissa
- [4] Normalize the result

$$X = A \times 10^3 = 0.9504 \times 10^3$$
$$Y = B \times 10^2 = 0.8200 \times 10^2$$

- 1) Compare exponents :
 $3 - 2 = 1$
- 2) Align mantissas
 $X = 0.9504 \times 10^3$
 $Y = 0.08200 \times 10^3$
- 3) Add mantissas
 $Z = 1.0324 \times 10^3$
- 4) Normalize result
 $Z = 0.10324 \times 10^4$



Floating Point Number Multiplication

- Steps:
 - Check for zeroes
 - Add the exponents
 - Multiply mantissa
 - Normalize the product
 - Notes:
 - *(Most Significant bit can not be 0)*
 - (we need normalization in the case of UnderFlow)
 - We do mantissa left shift by decreasing exponent by 1
 - For Example: $0.01011 \times 2^{12} \Rightarrow 0.10110 \times 2^{11}$

